

Q11. Design Scaler Academy Schema - RT **Attempted**

Design schema satisfying the following requirements mentioned below.

System requirements:

1. Students can register for different batches (Ex: Feb'20)
2. Students attend daily lectures on various topics (like, Trees, Linked lists, stacks etc.)
3. Every lecture has an assignment and homework.
4. Every assignment and homework can have 0-10 problems.
5. Every class has an instructor, some prelecture content and lecture notes.
6. Students should be able to see the class timeline, problem solve percentage, attendance percentage and status of all the problems in all the assignments/homework in the dashboard.
7. Instructors should be able to see a list of classes taken by him and his upcoming class.
8. Instructors should be able to create lessons for all the batches and should be able to add and modify the assignments and homework problems.

Future Scope:

1. Store ratings and feedback or the class.
2. Collaborative lectures taken by multiple instructors.
3. Daily Streak calculation.

NOTE:

- **Draw Class diagram and the ER diagram.**
- **Define the PKs for all the tables.**
- **Mention the columns which should be indexed.**

Rishi Create Schema Design

You've made a strong attempt, but there are some **design issues and missing relationships** that will cause problems (especially for dashboards, multi-batch registration, and tracking progress). I'll fix and extend your schema properly and then give:

1.  **Corrected ER design (with relationships)**
2.  **Class diagram (logical view)**
3.  **Primary Keys**
4.  **Indexes**
5.  **What needed fixing in your design**

✓ 1. Key Fix First (Important)

Your assumption:

Student → Batch (1:N)

✗ This is **incorrect per requirement**:

“Students can register for different batches”

👉 This means:

- **Student ↔ Batch = Many-to-Many (M:N)**

So we must introduce a **junction table**.

✓ 2. FINAL ER DIAGRAM (TEXTUAL)

Student (M) —< Student_Batch >— (M) Batch

Batch (1) —< Class >— (1) Instructor

Class (1) —< Lecture >— (M) LectureTopic >— (M) Topic

Lecture (1) —< Assignment >— (1) Assignment_Type

Assignment (1) —< Problem >— (M)

Student (1) —< Student_Problem_Status >— (1) Problem

Class (1) —< Attendance >— (M) Student

Instructor (M) —< Class_Instructor >— (M) Class (Future scope)

3. NORMALIZED TABLE DESIGN

◇ Student & Batch

```
Student(  
    student_id PK,  
    student_name  
)
```

```
Batch(  
    batch_id PK,  
    batch_name  
)
```

```
Student_Batch(  
    student_batch_id PK,  
    student_id FK,  
    batch_id FK,  
    enrolled_at  
)
```

◇ Instructor

```
Instructor(  
    instructor_id PK,  
    instructor_name  
)
```

◇ Class (Core Entity)

```
Class(  
    class_id PK,
```

```
    batch_id FK,  
    pre_lecture_content,  
    lecture_notes,  
    class_datetime  
)
```

◇ Instructor-Class Mapping (Future Ready)

```
Class_Instructor(  
    id PK,  
    class_id FK,  
    instructor_id FK  
)
```

👉 Supports:

- Single instructor (current)
- Multiple instructors (future)

◇ Lecture & Topics

```
Lecture(  
    lecture_id PK,  
    class_id FK,  
    lecture_name  
)
```

```
Topic(  
    topic_id PK,  
    topic_name  
)
```

```
Lecture_Topic(  
    id PK,  
    lecture_id FK,
```

```
    topic_id FK
)
```

◆ Assignment / Homework

```
Assignment_Type(
    type_id PK,
    type_name    -- Assignment / Homework
)
```

```
Assignment(
    assignment_id PK,
    lecture_id FK,
    type_id FK
)
```

◆ Problems (0–10 per assignment)

```
Problem(
    problem_id PK,
    assignment_id FK,
    problem_number
)
```

◆ Student Problem Tracking (VERY IMPORTANT)

```
Student_Problem_Status(
    id PK,
    student_id FK,
    problem_id FK,
    status,          -- Complete / Incomplete
    submitted_at
)
```

👉 Enables:

- Problem status
- Dashboard tracking
- Solve %

◇ Attendance

```
Attendance(  
  attendance_id PK,  
  student_id FK,  
  class_id FK,  
  status  
)
```

◇ Dashboard Metrics (Derived, not stored ideally)

Instead of storing:

- Problem_solve_percentage ❌
- Attendance_percentage ❌

👉 These should be **computed dynamically**.

◇ Instructor Features

```
Instructor_Lesson(  
  id PK,  
  instructor_id FK,  
  batch_id FK,  
  action,  
  assignment_id FK,  
  problem_id FK  
)
```

◇ Future Scope

Rating

```
Rating(  
    rating_id PK,  
    class_id FK,  
    student_id FK,  
    rating_value,  
    feedback  
)
```

Daily Streak

```
Daily_Streak(  
    id PK,  
    student_id FK,  
    streak_date,  
    count  
)
```

4. CLASS DIAGRAM (SIMPLIFIED)

Student

- student_id
- student_name

Batch

- batch_id
- batch_name

Class

- class_id
- class_datetime

Lecture

- lecture_id

Assignment

- assignment_id

Problem

- problem_id

Instructor

- instructor_id

Relationships:

Student *--* Batch

Batch 1--* Class

Class *--* Instructor

Class 1--* Lecture

Lecture 1--* Assignment

Assignment 1--* Problem

Student 1--* Problem (via status)

5. PRIMARY KEYS SUMMARY

Table	Primary Key
Student	student_id
Batch	batch_id
Student_Batch	student_batch_id
Class	class_id
Lecture	lecture_id
Assignment	assignment_id
Problem	problem_id
Instructor	instructor_id
Attendance	attendance_id
Student_Problem_Status	id

6. INDEXES (IMPORTANT FOR PERFORMANCE)

◇ Must Index

Student_Batch(student_id, batch_id)

Class(batch_id)

Class_Instructor(class_id, instructor_id)

Lecture(class_id)

Assignment(lecture_id)

Problem(assignment_id)

Student_Problem_Status(student_id, problem_id)

Attendance(student_id, class_id)

7. ISSUES IN YOUR ORIGINAL DESIGN

✗ 1. Student → Batch (wrong)

- Should be **Many-to-Many**

✗ 2. Assignment linked to Lecture incorrectly

- You used type instead of **separate Assignment entity**

✗ 3. Problem linked to type (wrong)

- Should be linked to **Assignment**

✖ 4. Class stores percentages

- Should be **computed, not stored**

✖ 5. Instructor_Classes incomplete

- Missing **Class FK**

✖ 6. No student progress tracking

- Needed **Student_Problem_Status**

FINAL TAKEAWAY

Your system now supports:

- ✓ Multi-batch enrollment
- ✓ Assignments & homework
- ✓ Problem tracking
- ✓ Dashboard metrics
- ✓ Instructor workflows
- ✓ Future scalability (ratings, multiple instructors, streaks)



